

ExfilTrack

A Forensically Sound USB Exfiltration Triage Tool

Project Proposal

Group Members:

Name	Index Number
Weerawarna S.B.M.N.	230699E
Thabrew D. C. L.	230631P
Dabarera G.D.M.	230111X

1. Introduction and Problem Statement

Modern software companies issue company-managed laptops to developers so they can build, test, and maintain proprietary source code, internal tools, and confidential client data. While this arrangement supports productivity, it also introduces a serious insider-threat risk: an employee with legitimate access to sensitive code and data can copy it to a personal removable storage device before leaving the organization, with no external attacker or network intrusion required.

This scenario commonly referred to as insider-driven data exfiltration via removable media, is difficult to detect after the fact. Even when a company laptop has no dedicated security or Data Loss Prevention (DLP) software installed, the underlying Windows operating system passively records substantial forensic evidence of device connections and file activity. However, this evidence is scattered across multiple independent system artifacts: the Windows Registry, Windows Event Logs (.evtx), shortcut files (.lnk), and Jump Lists. Investigating a suspected insider-theft incident currently requires a forensic analyst to manually extract and cross-reference each of these sources by hand, which is slow, error-prone, and inconsistent between investigators.

ExfilTrack addresses this problem by automating the collection, normalization, and correlation of these artifacts into a single, evidence-backed investigative timeline.

2. Motivation

Insider threats are consistently identified as one of the most damaging categories of security incident, precisely because the actor already has authorized access and no perimeter defense is triggered. Organizations need a way to reconstruct, after the fact, whether a departing or suspicious employee's use of removable storage is consistent with data exfiltration while being careful not to overstate what forensic artifacts can actually prove.

Our motivation for this project is threefold:

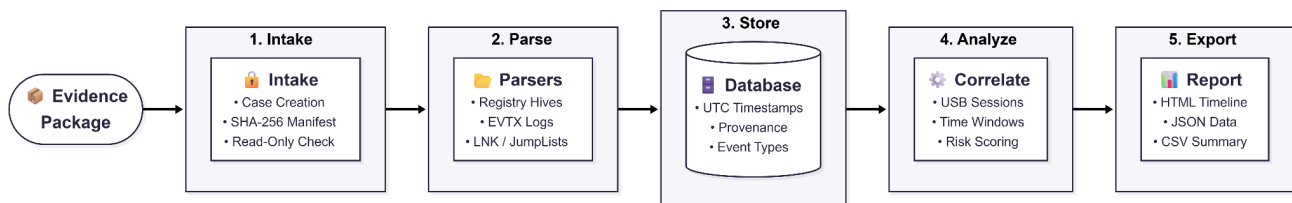
1. **Real-world relevance.** Insider data theft via USB drives is a well-documented and recurring risk for any organization that handles proprietary source code, trade secrets, or client data.
2. **Underserved tooling gap.** Existing open-source parsers (e.g., python-registry, python-evtx) can extract individual artifact types, but no lightweight, free tool correlates them automatically into a single triage report designed for non-specialist investigators (e.g., an internal IT security team without a dedicated forensics unit).
3. **Forensic responsibility.** A tool in this space must be built to avoid overstating its findings. Artifact proximity e.g., a file being accessed shortly after a USB is inserted is evidence consistent with exfiltration, not proof of it. ExfilTrack is deliberately designed around calibrated risk scoring and confidence levels rather than absolute claims, making it both more useful and more defensible than a tool that simply declares "theft confirmed."

3. Objectives

1. Build a Python-based tool that ingests offline Windows forensic evidence such as registry hives, .evtx exports, .lnk files, and Jump Lists in strict read-only mode.
2. Verify evidence integrity using SHA-256 hashing before and after analysis, preserving chain-of-custody.
3. Automatically identify USB storage devices, their connection timestamps, and reconstruct probable connection "sessions."
4. Correlate USB session activity with recent file-access artifacts on the same machine.
5. Apply an explainable, rule-based risk-scoring model (not a black box) to rank the likelihood of exfiltration-consistent activity.
6. Assign explicit confidence levels such as Low / Medium / High / Confirmed rather than binary "theft / no theft" conclusions.
7. Generate professional HTML, JSON, and CSV reports summarizing findings for a non-technical investigator or manager to review.

4. High-Level Approach and Architecture

ExfilTrack is structured as a five-stage offline pipeline:



4.1 Evidence Sources (Version 1 Scope)

Source	Purpose
Registry hives (SYSTEM, SOFTWARE, NTUSER.DAT)	Identify USB storage devices, vendor/product data, and device identifiers
Windows Event Logs (System.evtx, Security.evtx)	Detect storage-driver mount/unmount events
.lnk shortcut files	Extract target file paths, sizes, and access/modification timestamps
Jump Lists	Identify recently or frequently accessed documents per application

4.2 Correlation and Scoring Model

Rather than treating any single artifact as proof, ExfilTrack combines multiple weak signals into an explainable score. For example:

Indicator	Score Contribution
File activity within 30 seconds of USB insertion	+25
File activity within 5 minutes of USB insertion	+15
Sensitive file extension (.sql, .pem, .env, .zip)	+15

File located in a protected project directory	+20
Multiple confidential files accessed in one session	+15
Matching file found on destination USB (advanced feature)	+50

Each finding is also tagged with a **confidence level**:

- **Low** - a USB was connected, and a sensitive-looking file was accessed in the same general window.
- **Medium** - multiple related files were accessed shortly after insertion, consistent with staging behavior.
- **High** - supporting audit-log or destination-device evidence is present.
- **Confirmed** - a cryptographic hash match is found between the source file and a file on the destination USB.

This tiered model ensures ExfilTrack's output is phrased as "evidence consistent with possible exfiltration" rather than a definitive accusation, an important distinction for both technical accuracy and any downstream HR or legal use of the report.

4.3 Forensic Soundness Requirements

- Evidence files are never modified; all analysis output is written to a separate case directory.
- SHA-256 hashes are calculated before and after processing to prove no tampering occurred.
- Every parser records its name and version, and every finding cites its source artifact.
- Timestamps are preserved in original form and normalized to UTC for correlation.
- Malformed evidence produces an explicit error rather than being silently skipped.
- Running the same evidence through the same tool version produces identical results (reproducibility).

5. Threat Model: What ExfilTrack Can and Can't Detect

A key part of this project is being explicit about the boundary between what an **offline, post-incident forensic tool** can realistically achieve versus what requires organizational (IT) policy controls that sit outside the tool itself. We categorize exfiltration-related risks into three groups.

5.1 Category 1 - Addressed by Organizational IT Policy (Outside ExfilTrack's Scope)

Some exfiltration channels are best closed through standard IT administrative policy rather than forensic software, because they can be blocked with zero impact on legitimate developer work e.g., restricting personal mobile-device sync profiles or unauthorized removable-storage classes via Group Policy, and centralizing log forwarding. ExfilTrack's proposal documents these as **assumed organizational prerequisites** rather than something the tool itself implements, since they fall outside the scope of an offline forensic analysis tool.

5.2 Category 2 - Addressed by ExfilTrack's Forensic Parsers

Some suspicious behaviors rely on tools developers legitimately need every day (command-line utilities, local compilation, archive creation), so they can't be blocked externally without breaking normal engineering work. For these cases, ExfilTrack's value is in **post-incident detection**, not prevention: parsing filesystem change records and shell history to reconstruct what happened, without restricting what developers can do while working. This is the core of ExfilTrack's contribution and is addressed through the artifact parsers and correlation engine described in Section 4.

5.3 Category 3 - Documented Limitations

Consistent with good forensic practice, the proposal explicitly documents scenarios that **can't** be reliably detected by an offline artifact-analysis tool, so that ExfilTrack's findings are never overstated:

- Data copied via clipboard into approved communication or collaboration platforms, since this involves encrypted application traffic and volatile memory that is unavailable once the machine is powered off.
- Encrypted archive contents, since ExfilTrack can prove that an archive was created and moved to removable storage, but can't decrypt or verify the contents of a password-protected archive without the passphrase.
- Data exfiltrated purely through volatile memory or network sockets without ever touching non-volatile disk storage, since ExfilTrack is scoped to offline disk and registry artifacts and doesn't perform live memory or network capture.

Stating these boundaries clearly is itself part of responsible forensic tool design: a tool that claims to catch everything is less trustworthy than one that is precise about its actual evidentiary reach.

6. Implementation Plan

Technology stack:

Python 3, python-registry, python-evtx, standard library hashlib/json/csv, HTML templating for reporting.

Team division:

Member	Responsibilities
Member 1	Evidence intake, SHA-256 hashing/manifesting, Registry parsing
Member 2	EVTX parsing, .lnk/Jump List parsing, event normalization
Member 3	Correlation engine, risk/confidence scoring, report generation, evaluation

All members will jointly contribute to testing, documentation, and the final demonstration.

Testing approach:

Controlled scenarios will be generated using a free Windows virtual machine and personal USB flash drives, including

1. normal, non-suspicious USB use to measure false positives,
2. simulated theft of synthetic confidential files,
3. archive staging and deletion to test advanced correlation,
4. unrelated recent-file activity with no USB copy, to confirm the tool doesn't produce false accusations.

7. Deliverables

1. A working Python command-line tool implementing the evidence-intake, parsing, correlation, and reporting pipeline described above.
2. Sample case output including HTML, JSON, and CSV reports generated from test-scenario evidence.
3. A written final report documenting the tool's architecture, scoring model, evaluation results (detection accuracy, false-positive rate, reproducibility), and stated limitations.
4. This project proposal .
5. A presentation, covering the problem, approach, and a live or recorded demonstration of the tool against test evidence.

8. Timeline

Phase	Duration	Milestone
Proposal & design	1 Week	Finalize architecture, evidence schema
Core parser development	2 Weeks	Registry, EVTX, LNK, Jump List parsers working
Correlation engine	2 Weeks	Risk scoring, session reconstruction
Reporting & UI	2 Weeks	HTML/JSON/CSV report generation
Testing & evaluation	1 Week	Run controlled scenarios, measure metrics
Final report & presentation	1 Week	Documentation, rehearsal, submission

9. Limitations

ExfilTrack is an offline, post-incident artifact-analysis tool. It doesn't monitor systems in real time, doesn't intercept network or clipboard traffic, and can't decrypt password-protected data. Its conclusions are expressed as risk scores and confidence levels rather than definitive proof, reflecting the genuine evidentiary limits of the underlying Windows artifacts it analyzes. Where destination-device artifacts or cryptographic hash matches are available, the tool can provide stronger confirmation of file transfer; otherwise, its findings should be treated as leads for further investigation rather than conclusions.

10. Ethical Considerations

Because ExfilTrack analyzes artifacts that may include personal file names and USB device history, its use should be governed by organizational policy and applicable law regarding employee monitoring and data forensics. The tool is designed for use by authorized investigators on company-owned equipment as part of a defined incident-response process, not for covert or unauthorized surveillance.

11. Conclusion

ExfilTrack reconstructs USB-device activity, correlates it with relevant file-system and user-activity artifacts, and identifies evidence consistent with possible data exfiltration. Where destination-device artifacts or matching file hashes are available, the tool can provide stronger confirmation of file transfer. By combining an explainable risk-scoring model with clearly documented limitations, ExfilTrack aims to be both a practically useful triage tool and a forensically defensible one.